



University
of Exeter

Optimising the Locations of Police Vehicles Across Devon and Cornwall

Matt Trenchard

Background

This project involved developed a simulation to model police response times across Devon and Cornwall. By pairing this simulation with a genetic algorithm, we can find the most optimal location for each police car, in order to reduce response times.

Data Source

OSM (OpenStreetMap)

- Used to retrieve road networks and police base locations.
- Open source and free.
- Backed by major corporations (Microsoft, Meta, etc...)
- Community driven so there is potential for errors.

Police.uk

- Used to acquire the location and frequency of incidents that would require police attendance.
- Ran under the Open Government licence which means the data is official and free.

Processed Graph

- The data from OSM and Police.uk is read, processed and then combined in a weighted graph data structure.
- To improve the execution speed of the simulation, and to reduce storage space, the graph is pruned of many unnecessary nodes.

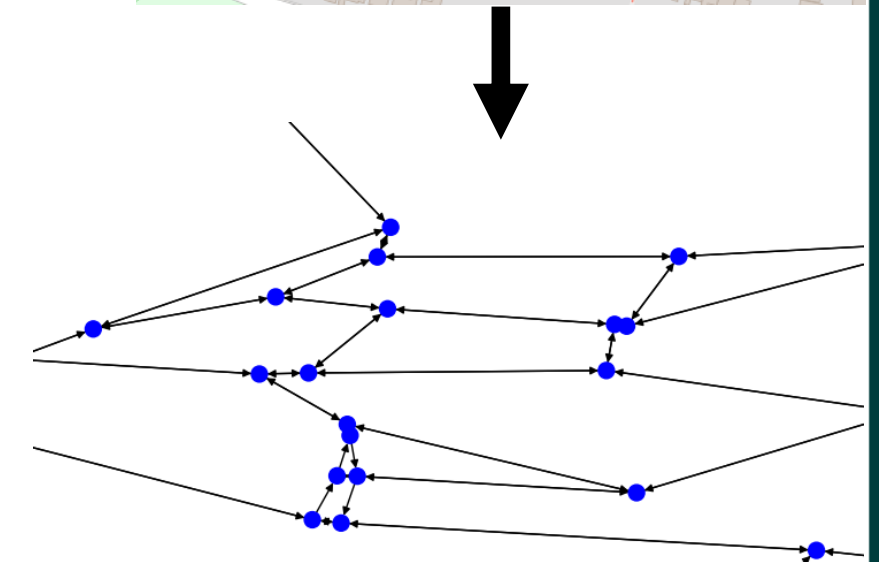
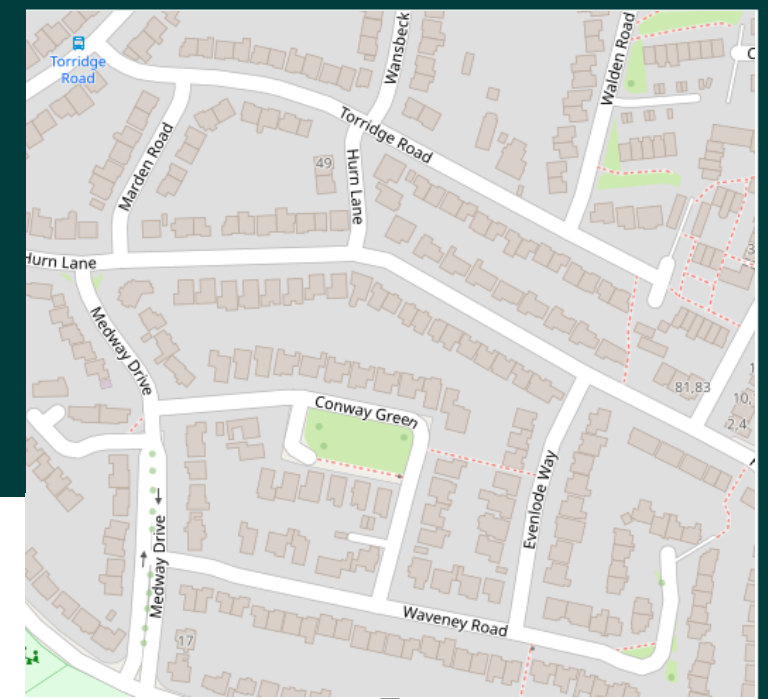


Figure 1: Shows the result of processing an OSM map.

Simulation

A discrete event simulation (DES) was developed which can be used to measure the performance of different vehicle positioning sets. The simulation is run on the processed graph structure and police responses are modelled by spawning incidents and then dispatching vehicles.

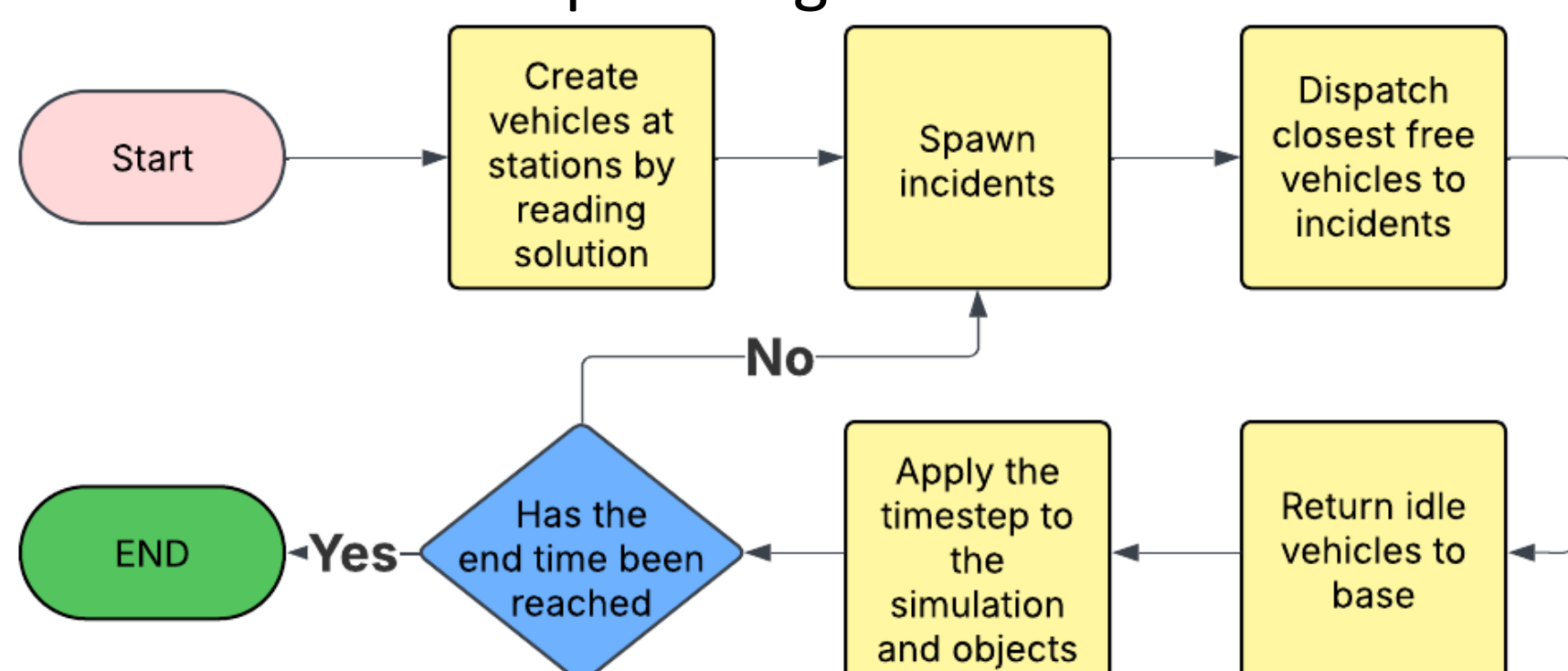


Figure 2: A flowchart of the simulation process.

One of the key goals of this project was to prove that simulation models can be both **realistic** and **efficient**. Therefore, many design choices were made to maximise the simulation's execution speed. This includes: using the Rust language, maximising use of direct access data structures, caching repeated calculations, and much more.

Genetic Algorithm (GA)

Genetic algorithms are based on the theory of natural selection. A large population of solutions is generated and then 3 key phases are repeatedly used to try to find optimal solutions. My GA represents solutions as a list of integers, where index n is the number of vehicles to be based at police station n .

- **Selection** – The process of choosing solutions based on their fitness score. In this case a solution's fitness is the average response time it produces when tested by the simulation, and selection is done via a tournament.
- **Crossover** – The process of creating new solutions by 'breeding' the selected ones. My GA uses n-point crossover, which involves the use of a random binary mask to combine two solutions.
- **Mutation** – The process of randomly altering solutions to explore new areas of the solution space. My GA does this by randomly reassigning 1 or more vehicle(s) to a different location.

Results

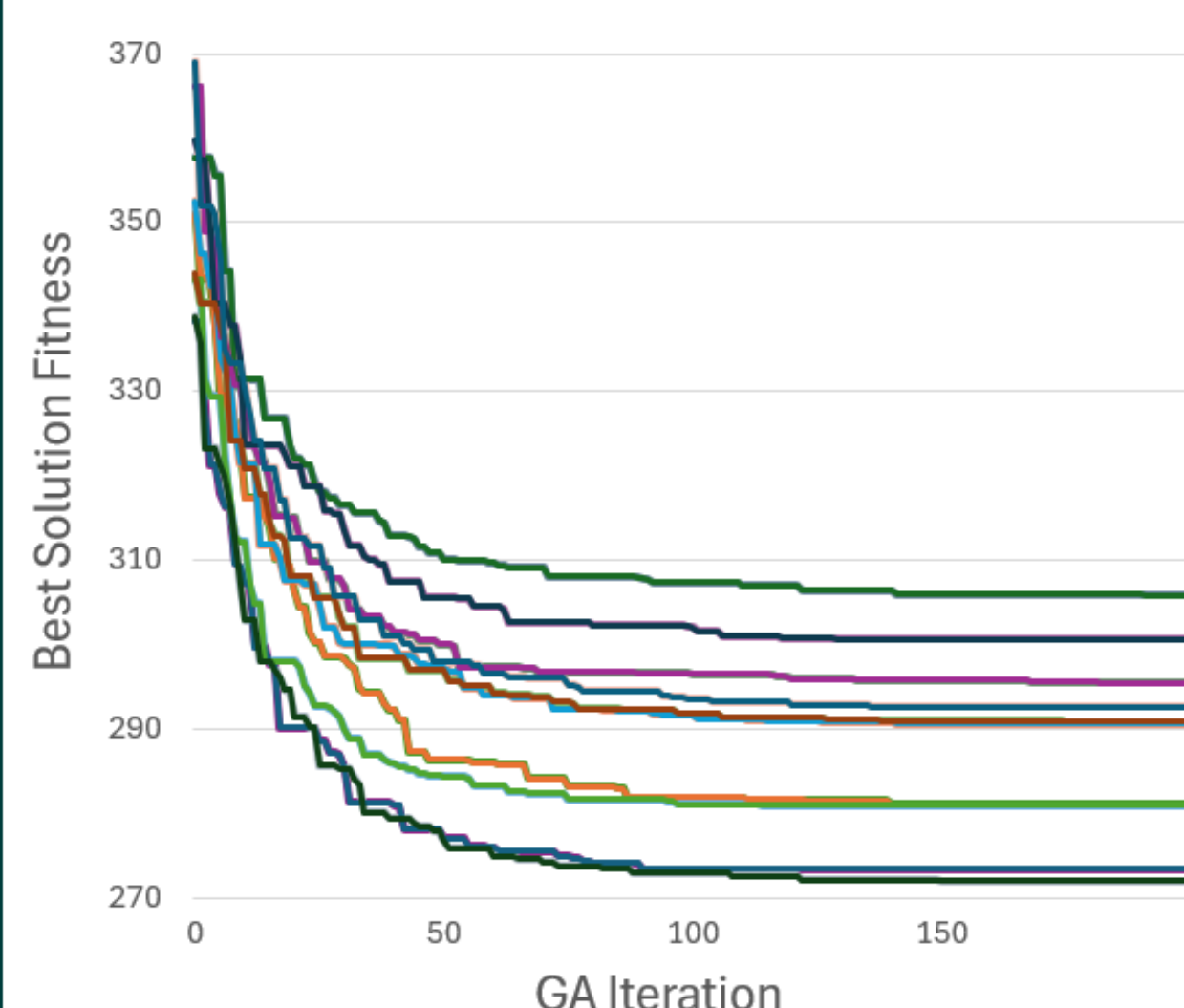


Figure 3: The fitness progression of the best overall solution from 10 independent GA runs.

- Combination of 10 independent GA runs, each training on a random spread of incidents.
- Simulations covered 72 hours with a 60 second timestep and used 100 vehicles.
- Similar solutions were consistently reached.
- The GA generally improved average response time by about 70 seconds (~20%).
- Other experiments (i.e. additional vehicles) showed potential for the future.

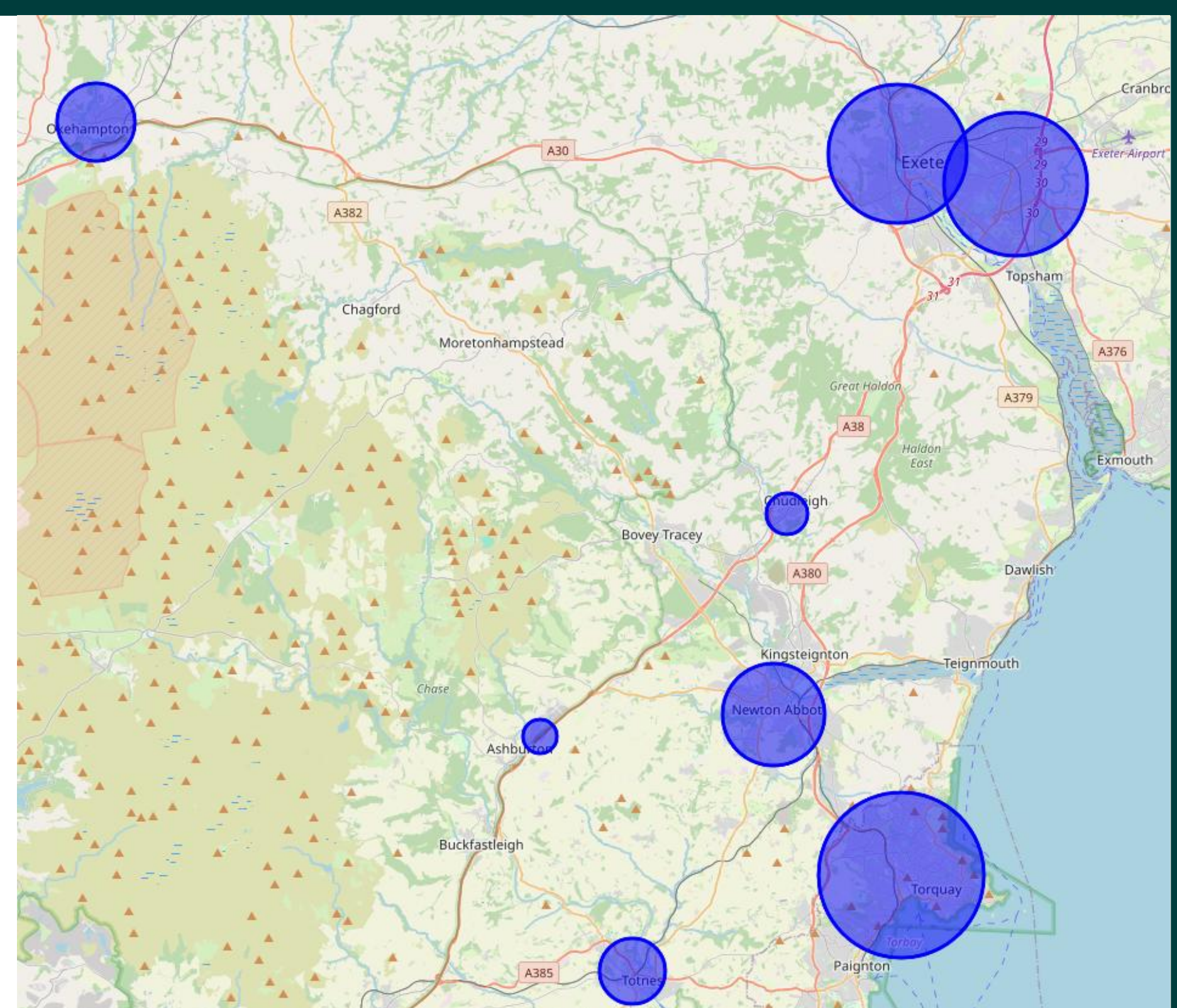


Figure 4: A subset of results. The size of each blue circle corresponds to the average number of police vehicles assigned to that station.